

METAMATEMATICO

Mapa del sistema

Nucleo Logico Evolutivo v7.0 + Lean 4 / Mathlib

Generado el 06/08/2026

Sobre este documento

Describe el sistema tal y como se comporta al ejecutarlo, no tal y como lo describe el paper. Cada cifra procede de una medicion sobre esta instalacion. Cuando un subsistema esta construido pero no influye en la salida, se dice explicitamente: es la diferencia entre 'el codigo existe y sus tests pasan' y 'cambia lo que el sistema responde'.

1. Que ocurre cuando escribes una consulta

```
Usuario (Streamlit, app.py)
|
v
Nucleo.process()
|
+--> Co-reguladores deciden la accion ..... CR_tac clasifica
|     RESPONSE (conversacion o matematica)
|     ASSIST    (el usuario adjunto codigo Lean)
|
+--> Es matematica? --NO--> LLM directo, sin tocar Lean
|
|     SI
|     v
|     _math_via_lean() ..... pipeline principal
|     1. LLM formaliza el enunciado en Lean 4
|     2. Se normaliza el codigo (ver seccion 3)
|     3. Lean verifica ..... FUENTE DE VERDAD
|     4. Si falla: reparar imports y reintentar
|     5. Si hay 'sorry': cascada de solvers
|     6. El LLM traduce el veredicto a lenguaje natural
|
+--> Se registra el resultado en la memoria MES
+--> Se alimenta el agente PPO
```

El principio de diseno es que la verdad matematica la establece Lean, no el modelo de lenguaje. El LLM cumple dos papeles distintos y separados: formalizador al principio y traductor al final. Entre ambos, quien decide es el verificador.

2. Estado real de cada subsistema

Un subsistema puede estar en tres situaciones distintas. Conviene no confundirlas.

Subsistema	Estado	Que hace hoy
LLM (Anthropic)	ACTIVO	Formaliza y traduce
Lean 4 + Mathlib	ACTIVO	Arbitro: acepta o rechaza pruebas
Normalizacion Lean	ACTIVO	Imports y lemas: sin ella casi nada compila
Reparacion de errores	ACTIVO	Localiza el modulo que falta y reintenta
Grafo de skills	ACTIVO	Inyecta skills y tactica en el prompt
Memoria procedimental	ACTIVO	Reutiliza tacticas que ya funcionaron
Co-regulador CR_tac	ACTIVO	Decide el enrutado de cada consulta

Cascada de solvers	PARCIAL	Solo entra si Lean devuelve 'sorry'
Otros co-reguladores	INERTE	Proponen, pero no alteran la decision
Colimites y patrones	INERTE	Construido y con tests; fuera del lazo
Evolucion / snapshots	INERTE	Construido; no toca la salida
E-equivalencia	INERTE	Construido; no toca la salida
GNN + PPO	DESACTIVADO	Red degenerada: misma accion para todo

Por que el GNN esta desactivado

La red se entreno con el objetivo 'todo problema matematico -> ASSIST'. Al haber una sola clase, aprendio la constante y alcanzo el 100% de acierto sin discriminar nada: devuelve ASSIST tambien para 'Hola' o 'Cual es la capital de Francia'. El sistema lo detecta con una sonda de cuatro entradas heterogeneas y, si colapsan a una sola accion, ignora la red y usa la heuristica. Recuperarla exige reentrenar con un objetivo que tenga al menos dos clases reales.

3. La capa que hace posible la verificacion

Un modelo de lenguaje escribiendo Lean falla casi siempre por tres motivos, y ninguno es matematico. Sin esta capa, practicamente ninguna prueba llegaba a compilar.

Problema	Tratamiento
import Mathlib completo	Se sustituye por un header estrecho
Imports inventados	Se validan contra las fuentes y se descartan
Lemas renombrados	Tabla de equivalencias (Nat.factors, etc.)
Modulo concreto ausente	Imports tematicos segun el contenido
Notacion con ambito	'open List' solo si el codigo usa ~
Identificador no hallado	Se busca en Mathlib y se reintenta

Por que no se permite 'import Mathlib'

Medido en esta instalacion: importar Mathlib entero tarda 742 segundos, frente a un limite de 360. Es decir, siempre expiraba y jamas devolvía nada. El header estrecho compila el mismo teorema en unos 11 segundos.

El bucle de reparacion

Cuando Lean falla, se extraen los identificadores del mensaje de error (Lean los nombra de cuatro formas distintas segun el contexto), se localizan sus declaraciones recorriendo los 7.746 ficheros fuente de Mathlib en una sola pasada (unos 2 segundos), se deduce el modulo de cada uno y se reintenta. El reintento solo se acepta si mejora el resultado.

Ejemplo real, area sin ninguna regla especifica:

- ```

1) Lean: ERROR - "Function expected at SimpleGraph"
2) Búsqueda en fuentes 2,0 s
 + import Mathlib.Combinatorics.SimpleGraph.Basic
3) Lean: SUCCESS 10,7 s

```

## 4. El grafo de skills

| Magnitud                  | Valor |
|---------------------------|-------|
| Nodos (skills)            | 172   |
| Morfismos                 | 553   |
| Skills fundacionales (L0) | 10    |

|                                 |     |
|---------------------------------|-----|
| Definiciones de dominio (L1-L3) | 162 |
| Categorías matemáticas          | 15  |
| Tácticas Lean                   | 9   |
| Estrategias de prueba           | 6   |
| Enlaces dominio -> táctica      | 165 |

Los niveles organizan la granularidad: L1 es el campo entero (teoría de grupos), L2 la sub-área (álgebra homológica) y L3 la sub-rama concreta (homomorfismos de grupos, factorización prima, teoría de Galois).

## Como llega el grafo a influir en la respuesta

Al recibir una consulta se buscan las skills que casan con ella, se recorren sus dependencias y su táctica asociada, y el resultado se anade al prompt del modelo. Las skills declaran términos en español y en inglés: sin ellos, ninguna consulta en castellano casaba con nombres escritos en inglés.

Consulta: "Demuestra el primer teorema de homomorfismo"

Contexto que recibe el LLM:

- relevant\_skills : group-homomorphisms
- prerequisites : group-theory
- suggested\_tactics : tactic-ring
- pillar : SET

Los enlaces dominio->táctica son morfismos de tipo TRANSLATION, no DEPENDENCY: las tácticas pertenecen al pilar de teoría de tipos y los dominios a conjuntos, categorías o lógica, de modo que el morfismo cruza pilares.

## 5. Rendimiento medido

| Operación                    | Antes | Ahora       |
|------------------------------|-------|-------------|
| Arranque en frío de Lean     | 552 s | 11 s        |
| Verificación de un teorema   | 167 s | 11-19 s     |
| Import de Mathlib completo   | 742 s | (prohibido) |
| Busqueda de un identificador | -     | 2 s         |
| Arranque de la aplicación    | -     | 4-8 s       |

### De donde viene la diferencia

Mathlib son 7.746 ficheros compilados. Cuando estaban en el disco USB, el coste no era el ancho de banda (39 MB/s medidos) sino la latencia de abrir miles de ficheros pequeños en un disco mecánico. Trasladar esa carpeta al disco de estado sólido redujo el arranque en frío de 552 a 11 segundos.

## 6. Como esta instalado

```
E:\Metamatematico\ proyecto (codigo, datos, paper)
|
+-- .lake --> enlace a C:\MetamatematicoLake (Mathlib compilado, SSD)
+-- app.py interfaz Streamlit
+-- nucleo/ el sistema
+-- .streamlit/secrets.toml clave de API (fuera de git)
+-- logs/ traza de ejecucion
+-- cache/ tmp/ todo lo temporal, nunca en C:
```

Acceso directo del escritorio

```
-> wscript -> Metamatematico.vbs -> Metamatematico.ps1
-> env_local.ps1 (redirige temporales) -> streamlit
```

El proyecto vive en E:. La única excepción deliberada es la carpeta de Mathlib compilado, que es cache de

compilacion y no datos del proyecto: se movio al SSD por rendimiento y se dejo un enlace, de modo que para Lean la ruta sigue siendo la misma.

## 7. Que no hace todavia

---

### Formalizar un marco teorico completo

El pipeline pide al modelo una unica formalizacion y Lean emite un unico veredicto sobre todo el fichero. Con cinco estructuras interdependientes, si cada una acierta el 70% de las veces, las cinco juntas salen el 17%. La via es descomponer: formalizar y verificar pieza a pieza, acumulando lo que ya compila y reparando lo que falla. El grafo aporta el orden de dependencias.

### Aprender de verdad

La memoria procedimental ya conserva que tactica funcione para cada consulta, pero el historico anterior se guardo sin ese dato y no es recuperable: hay que volver a sembrarlo. El agente neuronal necesita reentrenarse con un objetivo que discrimine.

### Reparar errores de tactica

La cascada de solvers solo actua ante 'sorry'. Los errores de compilacion, que son los que produce un modelo escribiendo Lean, aun no la activan salvo cuando se trata de un import ausente.

---

Metamatemático — Leonardo Jimenez Martinez (UNAM) · BIOMAT, Centro de Biomatemáticas. Suite de pruebas: 428 tests en verde en el momento de generar este documento.